

Something About C++

From ZZY

Something About C++

- 一、简介
- 二、代码风格
 - 1、Tab代码缩进
 - 2、大括号的问题
 - 3、空行的作用
 - 4、符号之间的空格问题
- 三、注意事项
 - 1、定义变量的问题
 - 2、初始化的问题
 - 3、数据范围
 - 4、与或非的表示
 - 5、数组越界问题
 - 6、if-else语句判断内容问题
- 四、常用技巧
 - 1、万能头文件
 - 2、定义常量方式
 - 3、if-else语句的另一种方式
 - 4、当测试样例为n时
 - 5、变量定义名常见方式
 - 6、求一个字符的ASCII码
 - 7、当多组样例时
- 五、如何Debug
 - 1、调试方法
 - 2、常见bug收集
 - (1) [Error] stary "\243" in program
 - (2) (output limit error)OLE
 - (3) 辅助输出的删除

一、简介

C++可以说是上大学以后最基础的语言了，如何写出一个好看的代码，如何代码少出错、不出错，如何运行时间快是个困扰无数人已久的的问题。

以下仅代表ZZY个人看法。

二、代码风格

好的代码风格可以让自己、让他人快速准确地读懂你的代码，所以要注意以下几个问题：

1、Tab代码缩进

代码缩进可以使代码结构整齐，也利于debug或者他人查看，一般来说，当某一段话被大括号括起来后，就要和上面的那行代码区别开，即用Tab进行缩进。例如以下代码：

```
int a=1000;
while (a>0){
    for (int i=1;i<=10;i++){
        a=a-i;
        if (a<=0){
            cout<<"Over"<<endl;
            break;
        }
    }
}
```

代码2-1 缩进示例

2、大括号的问题

就个人习惯来说，左大括号不换行，右大括号换行，即类似**代码2-1**：（else是否换行取决于个人习惯，本人不喜欢else换行）

左大括号是否换行因人而异。

如果左大括号也换行，当多个大括号嵌套的时候，会使得代码十分松散。

另外，如果大括号内只有一条语句，可以省略大括号，使代码变得简洁，就像**代码2-2**所示：

```
int main(){
    int n;
    cin>>n;
    int sum=0;
    for (i=1;i<=n;i++)//省略for循环的大括号
        sum=sum+i;
    //省略if和else的大括号
    if (sum>55) cout<<"Yes"<<endl;
    else cout<<"No"<<endl;
    return 0;
}
```

代码2-2 大括号的省略

3、空行的作用

一般来说，在代码之间加入空行是为了区别上下代码，在实现“输入-实现-输出”时可以在每块直接加空行。切忌滥用空行。

4、符号之间的空格问题

这个因人而异，当然加空格会好看很多，但是写代码时候不停敲空格也很难受，例如上面的代码2-3，修改后如下：

```
int a = 1000;
while (a > 0) {
    for (int i = 1; i <= 10; i++) {
        a = a - i;
        if (a <= 0) {
            cout << "Over" << endl;
            break;
        }
    }
}
```

代码2-3 符号间加了空格

确实要美观很多，对吧。如果养成写这种风格代码的习惯也挺好的。

三、注意事项

这一部分主要是写代码时候需要注意什么，需要考虑什么。类似于大括号匹配错误，少打了分号，等等错误在此不赘述。

1、定义变量的问题

C++如果在main函数外部定义变量，假设定义一个int变量，默认是0（其他可以类比）。但是如果在main函数里面定义变量，就得十分注意了。

在main函数之外的变量是全局变量，保存在静态储存区。而main函数里面的变量是局部函数，保存在栈区。一般来说在栈区要比静态储存区要小，所以需要定义很大的数组的时候，在main函数里面定义就有可能报错（因为空间不够）。

回到刚刚说的，例如在main函数里面定义了一个int变量x，且并未赋初始值的话，其x的值并非是0，而是一个不确定的数，这个值因电脑环境而异，可能在自己电脑上运行没问题，交上去就错了，所以一定要记得，要么在main函数外面定义，要么在内部定义的时候赋初始值(如果这个变量是输入数据，相当于输入后赋了值，无需之前赋值)。（例子见下）

```
#include <iostream>
using namespace std;
int sum; //默认初始值为0

int main(){
    for (int i=1;i<=10;i++){
        sum=sum+i;
    }
    cout<<sum<<endl;
    return 0;
}
```

代码3-1 全局变量

```
#include <iostream>
using namespace std;

int main(){
    int sum; //无默认初始值，可能会报错，应当int sum=0;
    for (int i=1;i<=10;i++){
        sum=sum+i;
    }
    cout<<sum<<endl;
    return 0;
}
```

代码3-2 局部变量定义注意事项

2、初始化的问题

当输入样例为多组时，必须记得将存储过数据或答案的变量、数组等清空，否则极易出错。清空时并非所有都需要清空，以以下代码为例：

```
/*
示例代码为，输出a+1，为了说明问题故意写复杂
下面程序很明显，a和b无需初始化，因为每次都被重新赋值覆盖了之前的值；但c必须在输入每个数据之后
初始化为0。
*/
int main(){
    int a;
    int b,c=0;
    while (cin>>a){
        if (a==0){
            break;
        }
        b=a+1;
        c=c+b;
        cout<<c<<endl;
    }
    return 0;
}
```

代码3-3 初始化注意事项

数组初始化方法为（以初始化为0为例）：`memset(a,sizeof a,0);`

3、数据范围

一定要看清楚题目的Hint，会给出一些数据的测试用例范围，如果可能超过int，存答案时候一定要开long long。

附：`int` 为32位，最大为2147483647，即大概 2×10^9 ，`long long` 为64位，最大为9223372036854775807，大概 9×10^{18} 。如果再大可能需要考虑高精度或者 `long long int` 等方式。

4、与或非的表示

要注意，`&&`，`||`，`!` 分别表示逻辑上面的与、或、非，用于判断语句等地方；`&`，`|`，`~` 表示位运算的与或非（就是高中学过的门电路），用于实现位运算的地方。

5、数组越界问题

C++的数组从0开始，务必记住！定义数组的时候尽量多往后定义一些，防止越界问题。比如输入n个数， $n \leq 100$ ，则定义数组建议定义到110左右。

如果习惯下标从1开始，此种定义方式也可以空出0位置不存任何数据。

6、if-else语句判断内容问题

- 不要在if语句内写逗号，例如 `a,b==0`，而应写成 `a==0 && b==0`；
- 不要在if语句内写连续的判断号，例如 `1<x<3`，而应写成 `x>1 && x<3`。
-

四、常用技巧

1、万能头文件

强烈不建议使用，`#include` 声明语句会包含官方提供/别人已经写好的部分包。

只需要万能头文件，`iostream`，`cstring` 啥的都不用写了！万能头文件为：`#include <bits/stdc++.h>`。不推荐的原因如下

- 初学者更应该清楚每个包是干什么的
- `include` 进来所有的包会造成命名空间的污染。举个例子，使用万能头文件会产生一些错误，你想命名一个名为 `queue` 的变量，如果你 `#include <bits/stdc++.h>` 可能编译器会提示你这个变量不可使用。
- 有的编译器不支持 `<bits/stdc++.h>`，提交上去会导致编译无法通过。

2、定义常量方式

将一个常量定义，这样后续代码只需使用该常量即可。注意常量不能被修改。定义方法为：

```
#define n 1000 或 const int n=1000;
```

建议使用后一种，因为 `const int` 会提供类型检查。

3、if-else语句的另一种方式

if-else语句可以使用简单方式代替：

`if (a) b else c` 等价于 `a?b:c`

4、当测试样例为n时

如果题目说先输入T，有T组测试样例时候，如何输入最方便呢？当然是如下方式！其中的道理不细述，大家自己思考即可。

```
int T;
cin>>T;
while (T--){
    .....
}
```

代码4-1 输入T组样例数据时

另外，若假设题目说“输入n个数，然后.....”，n需要在后文使用时，一般使用for循环输入：

```
int n;
cin>>n;
for (int i=0;i<n;i++){
    cin>>a[i]; //此为数组
}
```

代码4-2 输入n组数时

5、变量定义名常见方式

一般来说，变量名随个人喜好定义，但是切不可胡乱定义，否则代码一出错就容易找不出问题，还不容易让他人看懂代码。

这些表示因人而异。

`maxn`：n的最大值，一般用来定义常量

`T`：测试用例个数

`cnt`：count的缩写，当作计数器之类的用途

`sum`：求和的结果

`flag`：一般设置为bool类型，用于判断等

`ch`：一般设置为char类型

`str`：一般设置为string类型

`i, j, k`：一般用于循环

6、求一个字符的ASCII码

```
int i='a';//表示字符a的ASCII码
char c=48;//表示ASCII码为48的字符
```

代码4-3 求ASCII码

7、当多组样例时

如果以0 0结尾时，代码这样写：

```
int a,b;
while (cin>>a>>b){
    if (a==0 && b==0) break;
    .....
}
```

代码4-4 以0 0结尾时

当多组样例直到文件结尾时：（此时自己在电脑上运行时输入了所有数据后并不会停止，属正常情况）

```
int a,b;
while (cin>>a>>b){
    .....
}
```

代码4-5 直接结尾时

五、如何Debug

1、调试方法

很多时候，都不要怀疑是题目出错了或者是数据出错了，因为这个几率真的真的不大！当你出错了以后，你应该自己解决，不能过多依赖别人帮忙，这样取得不了进步。你应该：

- 再仔细看一下题，有没有读错题
- 一行行认真查看代码，检查是否有字母打错、输出大小写出错、少了空格或者句号等等
- 自己造极端数据来尝试着卡自己的代码
- 查看题目给的数据范围，想想是否越界
- 把代码中的中间变量输出出来检查，看看是否每一步都如你所想的那样进行
- 耐心耐心再耐心！

2、常见bug收集

(1) [Error] stary "\243" in program

这个bug是说你的程序中出现了中文字符。比较常见的是出现了 `；`、`,`、`,`、`,`、`,` 这类的全角中文标点，偶尔中文注释或者字符串中的中文也会导致这种情况。

(2) (output limit error)OLE

输出了过多的数据。举一个简单的例子，以下代码块：

```
while(true){  
    cout << "hello world" << endl;  
}
```

就会不断输出"hello world"。提交就会产生OLE错误。在编程过程中，一般将 `cout` 语句加在死循环中，而死循环又没有正确地结束，会出现这种情况。

(3) 辅助输出的删除

如果在代码中，如果为了调试方便，加入了类似 `cout << "请输入一个整数：" << endl;` 之类的提示性语句，在提交代码时记得删除。

(未完待续)